CS108 Project – Spring Semester

Wormhole: A Mini Game Hub in Python

Built with Pygame

Mohit Agrawal Arush Anand

April 29, 2026

Abstract

This report aims to help the reader access and use our project – *Wormhole* – easily, without requiring extensive comprehension of the source code. It covers the project's concept, design, implementation, and the challenges faced during development. Wormhole is a two-player themed game hub built using the `pygame` library in Python, featuring a pixel-art knight who navigates a richly themed menu across three distinct worlds, each hosting classic two-player board games.

Contents

1 Introduction **3**

2 Project Overview **3**

3 System Modules **3**
3.1 External Libraries . . . 3

4 Project Structure **4**

5 Major Features **4**
5.1 Medieval Fantasy Menu . . . 4
5.2 Theme System . . . 5
5.3 Board Games . . . 5
5.4 Leaderboard and Statistics . . . 5

6 Important Functions **5**
6.1 `game.py` – Base Game Class . . . 5
6.2 `tictactoe.py` – TicTacToe . . . 6
6.3 `connect4.py` – Connect4 . . . 6
6.4 `othello.py` – Othello . . . 6
6.5 `Menu.py` – Main Menu . . . 7
6.6 `Level.py` – World Level . . . 7

6.7 `Character.py` – Player Character . . . 7
6.8 `Settings.py` – Settings Panel . . . 7
6.9 `Game_Over.py` – Post-Game Screen . . . 7
6.10 `stats.py` – Statistics . . . 8

7 Logic and Algorithms **8**
7.1 Inheritance-Based Game Architecture . . . 8
7.2 Board Representation with NumPy . . . 8
7.3 Win Detection . . . 8
7.3.1 TicTacToe – Five in a Row . . . 8
7.3.2 Connect4 – Four in a Row with Gravity . . . 9
7.3.3 Othello – Directional Flanking . . . 9
7.4 Gravity Simulation (Connect4) . . . 9
7.5 Camera and Tile-Based Map . . . 9
7.6 Region and Zone System . . . 9
7.7 Settings State Machine . . . 9
7.8 Post-Game Data Flow . . . 9
7.9 Animation System . . . 10

8 Running Instructions **10**
8.1 Prerequisites . . . 10
8.2 Virtual Environment Setup . . . 10
8.3 Launching the Game . . . 10
8.4 Controls . . . 10

9 Challenges **10**

10 Future Work **11**

11 Conclusion **11**

1. Introduction

Wormhole is a two-player game hub developed as part of the CS108 course project. The project draws inspiration from retro RPG aesthetics and classic board games, combining them into a single cohesive experience. Players register accounts, navigate a pixel-art world, and compete across three well-known board games: Tic-Tac-Toe, Connect 4, and Othello.

The game hub is designed to be extensible. Its modular theme system allows different visual identities – from medieval fantasy to dystopian science fiction – to be applied seamlessly across the menu, characters, boards, and tokens.

2. Project Overview

Players begin by registering accounts through a terminal-based system. Credentials (username and password) are stored in a `.tsv` file, enforcing uniqueness per user.

Once registered, players enter the **Menu World** where a pixel-art knight serves as the navigation cursor. The knight can enter one of three themed *worlds*:

- **Kingdom of Heaven** – A medieval fantasy setting with wooden textures and stone boards.
- **Blade Runner 2049** – A neon-lit cyberpunk world with holographic boards.
- **Dune** – A desert world with rocky terrain and sand-toned boards.

Within each world, two players are assigned character roles (protagonist vs. antagonist) and compete in their selected game. Non-static sprites indicate whose turn it is. Players may either make a move or resign.

After each match, players encounter **The Director** – an omniscient arbiter who presents a sortable leaderboard (by wins, losses, or win/loss ratio) and post-match statistics.

3. System Modules

3.1 External Libraries

The project depends on the following external Python libraries:

Library	Purpose
`pygame-ce` [3]	Community edition of Pygame; core game rendering and event loop
`numpy` [2]	Efficient array operations for game-state representation
`matplotlib` [1]	Static and interactive visualisations for the statistics screen
`os`	OS-level file and path operations
`sys`	Interpreter interaction and exit handling
`subprocess`	Spawning shell scripts (e.g. the leaderboard script)

4. Project Structure

```
.
|-- hub/
|   |-- Assets/
|   |   |-- Game_Over/
|   |   |   |-- Background.png
|   |   |   `-- Fonts.ttf
|   |   |-- Images/
|   |   |-- Fonts/
|   |   `-- Theme.py
|   |-- games/
|   |   |-- connect4.py
|   |   |-- othello.py
|   |   `-- tictactoe.py
|   |-- Menu_Assets/
|   |   |-- Codes/
|   |   |-- Csv_Files/
|   |   |   |-- Map_Boundary.csv
|   |   |   `-- Map_Regions.csv
|   |   |-- Fonts/
|   |   `-- Images/
|   |-- Game_Over.py
|   |-- game.py
|   |-- Menu.py
|   |-- stats.py
|   |-- history.csv
|   |-- users.tsv
|   |-- leaderboard.sh
|   `-- main.sh
|-- Report/
|   |-- Makefile
|   |-- Report.tex
|   |-- Bibliography.bib
|   `-- Report.pdf
|-- .gitignore
|-- README.md
`-- Requirements.txt
```

5. Major Features

5.1 Medieval Fantasy Menu

The menu presents a pixel-art world rendered using `pygame-ce` [3]. Players control a knight character using `WASD` keys, interacting with in-world NPCs (spearmen) via the `E` key to trigger game selection, speed settings, and theme selection.

5.2 Theme System

Three thematic skins are available, each affecting menu visuals, character sprites, game boards, tokens, and sprite animations. The pixel art assets were sourced from the Tiny Swords asset pack [4] and supplemented with AI-generated artwork.

Kingdom of Heaven Wooden textures, stone boards, Knight and Archer characters.

Blade Runner 2049 Neon palette, holographic boards, Roy and Decker characters.

Dune Rock and sand aesthetics, desert boards, Paul Atreides and Harkonnen characters.

5.3 Board Games

- **Tic-Tac-Toe**: 10×10 board; first to achieve 5 in a row wins.
- **Connect 4**: 7×7 board; standard drop-piece mechanics [**?**].
- **Othello**: 8×8 board; standard flipping rules [**?**].

During gameplay, animated non-static sprites indicate the active player. Either player may resign on their turn.

5.4 Leaderboard and Statistics

The Director presents a terminal-rendered leaderboard sortable by total wins, total losses, or win/loss ratio. Post-match statistics include per-game popularity metrics and top-winner summaries, rendered via `matplotlib` [1].

6. Important Functions

This section documents the key functions defined across all Python modules in the project.

6.1 `game.py` – Base Game Class

`Game.__init__(self, game_name, players, theme, Characters)`
Initialises Pygame [3] and all shared game state: screen surface, clock, current player index, resolution, asset theme, and the tie flag. Every concrete game (TicTacToe, Connect4, Othello) calls this via `super()`.

`Game.generate_background(self)`
Blits a resolution-scaled background image to the display and captures two sub-surface snapshots at each character's position. These snapshots are reused each frame to restore the background behind animated sprites, preventing overdraw artefacts.

`Game.generate_players(self)`
Renders the two character sprites with a simple two-frame toggle animation. Only the active player's sprite animates; the idle player is frozen on frame 0. Sprites for player 2 are horizontally flipped so both characters face the board.

`Game.Resign(self)`
Draws the resign button and detects a left mouse-button press inside its hit-rectangle. On activation it sets `self.running = False` and calls `switch_turns()` so the resigning player becomes the loser when the result is reported.

`Game.redraw_tokens(self)`
Iterates over the entire board array [2] and blits the appropriate scaled token sprite for every occupied cell. Cells holding `-1` (empty) are skipped.

`Game.run(self)`
The main game loop: ticks the clock at 60 FPS, dispatches Pygame events to `event_handler()`, calls all draw helpers, and calls `pygame.display.update()`. Returns a `(tie, current_player)` tuple when `self.running` becomes `False`.

6.2 `tictactoe.py` – TicTacToe

`TicTacToe.win_check(self)`
Checks the last-placed token for five in a row, column, main diagonal, or anti-diagonal using NumPy [2] slice comparisons against a checker array of five equal values.

`TicTacToe.update_board(self)`
Places a token on the board array at `current_move` if the cell is empty, redraws all tokens, then calls `win_check()`. Sets `self.running = False` on a win or a full board (tie), otherwise calls `switch_turns()`.

`TicTacToe.event_handler(self, event)`
Converts a mouse-button-down event into board grid coordinates by scaling pixel position against the board boundaries and cell dimensions, then calls `update_board()`.

6.3 `connect4.py` – Connect4

`Connect4.win_check(self)`
Uses NumPy [2] slice comparisons to scan every window of four consecutive cells in the row, column, main diagonal, and anti-diagonal through the last-moved position. Returns `True` as soon as any window matches the current player's token.

`Connect4.update_board(self)`
Extracts the clicked column, finds the lowest empty row using `numpy.where`, places the token there (gravity simulation), then checks for win or tie. If neither condition is met, turns are switched.

`Connect4.event_handler(self, event)`
Determines the column index from the horizontal pixel offset of a mouse click, ignoring clicks outside the board boundaries, then delegates to `update_board()`.

6.4 `othello.py` – Othello

`Othello.check_valid(self, x, y, player)`
Returns `True` if placing a token at `(x, y)` is legal per the official Othello rules [**?**]: the cell must be empty, and at least one direction must have a contiguous sequence of opponent pieces ending at a friendly piece.

`Othello.valid_moves(self, player)`
Iterates every board cell and calls `check_valid`; draws a hint token on each legal cell so the current player can see available moves.

`Othello.flip_pieces(self, x, y, player)`
For each of the eight directions, collects opponent pieces that would be sandwiched by the new move and flips them to the current player's colour, then places the new piece.

`Othello.win_check(self)`
Sets `self.end = True` when the board is completely full or when neither player has any legal move remaining [**?**].

`Othello.update_board(self)`
After a valid move, calls `flip_pieces()`, switches turns, and handles the pass-or-end

scenarios: if the next player has no moves the turn reverts; if neither player has moves the game ends.

`Othello.counter(self)`
Counts how many cells each player occupies using `numpy.sum` [2] and renders the totals as on-screen score labels scaled to the current resolution.

6.5 `Menu.py` – Main Menu

`Menu.run(self)`
The top-level application loop. Runs the Level at 60 FPS, detects which game was selected by `level.settings.current_game`, launches the appropriate game class, writes the result to `history.csv`, and then calls `Game_Over` and `stats_main()` in sequence.

6.6 `Level.py` – World Level

`Level.camera_draw(self, group)`
Computes a camera offset so the player sprite stays centred on screen, clamps the offset to the map boundaries (4×1280 wide, 4×720 tall), and blits every sprite in the group with the offset applied.

`Level.create_boundary(self)`
Reads `Obstacle_Map` (a 2-D list parsed from CSV) and spawns a `Tile` sprite for every non-`-1` cell, building the static collision layer.

`Level.create_regions(self)`
Reads `Region_Map` and assigns each tile to one of four sprite groups (Spawn, Settings, Game Selection, Hinterland) used for region-change detection.

6.7 `Character.py` – Player Character

`Player.move(self, speed)`
Normalises the direction vector, advances the hitbox, and checks collisions before committing the move. This separation of hitbox and display rect allows tight collision geometry independent of sprite size.

`Player.animate(self)`
Increments an animation index at 0.15 per frame and selects the correct sprite frame for the current status (`right_idle`, `left_move`, etc.), flipping the image for leftward movement.

`Player.check_zones(self)`
Iterates the `Interactive` zone list and returns `True` if the hitbox overlaps any zone rectangle, storing the matched zone for the Settings panel.

6.8 `Settings.py` – Settings Panel

`Settings.run(self)`
Reads `pygame.key.get_just_pressed()` each frame and branches on `self.zone["Zone"]` to render one of: speed adjustment, menu-character selection, user character selection, theme selection, game-rules display, or leaderboard/exit options. Sets `self.current_game` to signal `Menu.py` which action to take.

6.9 `Game_Over.py` – Post-Game Screen

`Game_Over.draw(self)`
Renders the result message, a 4×3 grid of selectable sort-criterion cells (game $\times$ stat type), and two buttons: SHOW (enabled only after a criterion is selected) and Next.

`Game_Over.call(self, key)`
Invokes `leaderboard.sh` via `subprocess.run`, passing the history CSV, users TSV, and the two-part sort key (column index and stat index) as command-line arguments.

`Game_Over.run(self)`
Event loop that handles cell selection by index, triggers `call()` on SHOW or Enter, and exits on Next.

6.10 `stats.py` – Statistics

`read_history()`
Parses `history.csv` line by line into a list of dicts with keys `Winner`, `Loser`, `Date`, and `Game`, skipping any malformed rows.

`stat_calculation(data)`
Aggregates wins per player into a dict and collects all game names into a list, which are the two data sources needed for the charts.

`graphs(wins, games)`
Produces a transparent-background bar chart of the top-5 players and a pie chart of game-play distribution using `matplotlib` [1], saving both as PNG files consumed by the Pygame stats screen.

`stats_main()`
Orchestrates the full stats pipeline: reads history, calculates stats, generates charts, and then runs the Pygame [3] display loop that shows the histogram and pie chart side-by-side with a Next button.

7. Logic and Algorithms

7.1 Inheritance-Based Game Architecture

All three board games inherit from the abstract `Game` class in `game.py` (see Section 6.1). The base class implements the render pipeline, resign detection, player animation, and the main loop. Each subclass overrides only `win_check()`, `update_board()`, and `event_handler()`, so shared behaviour is never duplicated. This approach follows standard Pygame game design practice [**?**].

7.2 Board Representation with NumPy

Every board is stored as a 2-D NumPy [2] integer array initialised to `-1` (empty). Player 0's tokens are stored as `0` and player 1's as `1`. This representation enables vectorised slice comparisons for win detection and `numpy.where` for gravity in Connect4, avoiding explicit Python loops over the grid [**?**].

7.3 Win Detection

7.3.1 TicTacToe – Five in a Row

After each move at `(x, y)`, six overlapping windows of length five are checked along the row, column, main diagonal (`numpy.diagonal(offset = y - x)`), and anti-diagonal (`numpy.diagonal` of the horizontally flipped board with `offset = 9 - x - y`). A window matches if it equals a five-element checker array of the current player's value. See Section 6.2 for the corresponding function documentation.

7.3.2 Connect4 – Four in a Row with Gravity

The same sliding-window technique is applied with a window length of four on a 7×7 grid [?]. Gravity is implemented in `update_board`: the clicked column is extracted, `numpy.where(col == -1)` finds empty rows, and the token is placed at the last (lowest) index, making tokens fall to the bottom. See Section 6.3.

7.3.3 Othello – Directional Flanking

A move at `(x, y)` is valid only if at least one of the eight directions has a contiguous run of opponent pieces ending at a friendly piece (sandwiching), per the official rules [?]. `check_valid` implements this traversal. `flip_pieces` then re-traverses each valid direction and converts all sandwiched opponent cells to the current player's colour. `win_check` declares the game over when the board is full or neither player can move. See Section 6.4.

7.4 Gravity Simulation (Connect4)

Gravity is purely data-driven: no physics simulation is needed. When a column is clicked, `numpy.where(col == -1)[0]` returns the indices of all empty rows. Selecting the last element of this array (highest index = lowest visual row) replicates the effect of a token falling to the bottom of the column.

7.5 Camera and Tile-Based Map

The menu world is a 5120×2880 pixel map ($4\times$ the base resolution of 1280×720). The camera offset is computed each frame as the difference between the player's centre and half the screen dimensions, then clamped so the camera never shows the void outside the map. Collision and region tiles are loaded from CSV grids into Pygame [3] sprite groups; collision checks use `pygame.Rect.colliderect` between the player's hitbox and each tile's rect.

7.6 Region and Zone System

Four region sprite groups (Spawn, Settings, Game Selection, Hinterland) mirror a `Region_Map` CSV. Each frame the player's hitbox is tested against every region group to update `Player.Region`, which controls the banner display. Interactive zones are defined as simple rect dictionaries in `Support.py`; `check_zones()` tests the hitbox against these rects and stores the matched zone so the Settings panel knows what to render.

7.7 Settings State Machine

The Settings panel acts as a lightweight state machine. Its single `run()` method reads the active zone name and branches into mutually exclusive rendering and input paths: speed control, menu character, user character selection, theme, game rules, leaderboard, and exit. State is communicated back to `Level` and `Menu` via plain instance variables (`current_game`, `theme`, character indices), avoiding callbacks or event queues.

7.8 Post-Game Data Flow

On game end, `Menu.run()` writes a row to `history.csv` (winner, loser, timestamp, game name). `Game_Over.run()` lets the user select a sort criterion, then calls `leaderboard.sh` via `subprocess.run` with the criterion encoded as two integer arguments. `stats_main()` then reads the same CSV, computes aggregates, renders transparent-background Matplotlib [1] charts saved as PNGs, and displays them in a Pygame window – creating a complete data pipeline from raw CSV to in-game visualisation.

7.9 Animation System

Both the menu character and in-game character sprites use a float counter incremented by 0.15 per frame. The floor of this counter modulo the frame count selects the current animation frame, producing a smooth approximately 6-frame-per-second animation at 60 FPS. The active player's sprite cycles between two frames; the inactive player is frozen. The player-2 sprite is flipped horizontally so both characters face the board regardless of screen position.

8. Running Instructions

8.1 Prerequisites

- Python 3.x installed
- `numpy` [2], `matplotlib` [1], and `pygame-ce` [3] installed

8.2 Virtual Environment Setup

```
python -m venv venv
source venv/bin/activate
python -m pip install -r requirements.txt
```

8.3 Launching the Game

```
bash ./hub/main.sh
```

8.4 Controls

Key	Action
`W A S D`	Move the knight
`E`	Interact with NPCs / select options

Note: Speed is uncapped by design – exploring out-of-bounds is an intentional easter egg.

9. Challenges

- **Theme switching at runtime**: Dynamically reloading sprite sheets and board assets across three fully distinct visual themes without restarting the game loop required careful asset management and lazy loading strategies.
- **Tile-based map integration**: CSV-driven map boundaries and region definitions had to be parsed and rendered efficiently to maintain smooth frame rates. The unavailability of pyTMX required a custom approach using a `Map.png` and CSV files for map loading and interaction handling.
- **Cross-process communication**: Bridging the Python game loop with the Bash leaderboard script via `subprocess` required careful handling of I/O streams and process lifecycle.
- **Pixel art generation**: Creating cohesive sprite sets for three distinct themes involved leveraging AI tools alongside the Tiny Swords asset pack [4].
- **Resizable windows**: All images needed to be carefully scaled using variables tied to the current screen resolution.

10. Future Work

This section outlines planned improvements and extensions that would be implemented given an additional ten hours of development time.

- Add Sound systems
- Add an Easter Egg in the Hinterlands(Originally intended but could not due to time limit)
- Add a game Inscryption
- Smoothen out Animations during gameplay
- More Menu characters that player can control

11. Conclusion

Wormhole successfully demonstrates a modular, theme-driven game hub architecture in Python [**?**]. The project integrates a real-time pixel-art navigation menu, three fully-playable two-player board games, persistent user accounts, and a statistical leaderboard system. The codebase is structured to make adding new games or themes straightforward. For further implementation details, refer to Section 6 and Section 7.

References

[1] Matplotlib Official Documentation. https://matplotlib.org/stable/index.html, 2025. Accessed: 2025.

[2] NumPy Official Documentation. https://numpy.org/doc/, 2025. Accessed: 2025.

[3] Pygame CE (Community Edition) Official Documentation. https://pyga.me/docs/, 2025. Accessed: 2025.

[4] Pixel Frog. Tiny Swords Asset Pack. https://pixelfrog-assets.itch.io/tiny-swords, 2025. Accessed: 2025.